

Discrete Fourier Transform: Physical Coordinates, Discrete Coordinates, and Array Positions

In the continuous Fourier transform, a signal is described as a function of continuous time t , and its Fourier transform is described as a function of continuous angular frequency ω :

$$b(t) \longleftrightarrow a(\omega).$$

When the problem is discretized, we replace the continuous coordinates t and ω by discrete coordinates. We define

$$n = \frac{t}{\Delta t}, \quad m = \frac{\omega}{\Delta \omega}.$$

Equivalently,

$$t_n = n\Delta t, \quad \omega_m = m\Delta \omega.$$

Therefore,

$$b_n = b(t_n) = b(n\Delta t),$$

and

$$a_m = a(\omega_m) = a(m\Delta \omega).$$

The important point is that n and m should not be thought of primarily as positions in a computer vector. They are **discrete coordinates** obtained from the continuous physical coordinates t and ω .

A computer must also store the values b_n and a_m in arrays. We will use different symbols, k and j , for the corresponding **array positions**.

Thus we distinguish

physical coordinate	discrete coordinate	array position
t	$n = t/\Delta t$	k
ω	$m = \omega/\Delta \omega$	j

This distinction becomes especially important for the Fourier coefficients, because the order in which the FFT stores frequencies is not the same as a simple increasing physical frequency axis.

Discrete time axis

For N time samples separated by Δt ,

$$t_n = n\Delta t.$$

For the usual DFT convention,

$$n = 0, 1, \dots, N - 1.$$

Therefore,

$$b_n = b(n\Delta t).$$

For example,

$$b_0 = b(0), \quad b_1 = b(\Delta t), \quad b_2 = b(2\Delta t), \dots$$

Here n is the discrete time coordinate.

In Python and C, the array positions also start at zero, so in this simple case the array position and the discrete coordinate happen to be the same:

$$k = n.$$

In MATLAB, array positions start at one, so

$$k = n + 1.$$

For example, the mathematical value b_0 is stored as

$$\mathbf{b}[0]$$

in Python or C, but as

$$\mathbf{b}(1)$$

in MATLAB.

This difference is only a matter of array storage. It does not change the mathematical definition of n .

Discrete frequency axis

For N time samples with sampling interval Δt , the angular-frequency spacing is

$$\Delta\omega = \frac{2\pi}{N\Delta t}.$$

The discrete frequency coordinate is defined by

$$m = \frac{\omega}{\Delta\omega},$$

or equivalently,

$$\omega_m = m\Delta\omega.$$

The Fourier coefficient associated with that frequency is

$$a_m = a(\omega_m).$$

Again, m represents the discrete frequency coordinate, not necessarily the position where a_m is stored in an FFT output vector.

The Discrete Fourier Transform

Using these discrete coordinates, the DFT is

$$a_m = \sum_{n=0}^{N-1} b_n e^{-i2\pi mn/N}.$$

The inverse transform is

$$b_n = \frac{1}{N} \sum_m a_m e^{i2\pi mn/N}.$$

Here,

$$n = \frac{t}{\Delta t}$$

represents the discrete time coordinate, while

$$m = \frac{\omega}{\Delta\omega}$$

represents the discrete frequency coordinate.

The symbols n and m belong to the mathematical description of the discretized problem. They should be distinguished from the positions where the corresponding values are stored in a computer array.

Why the distinction matters for the FFT

Consider an even number of samples, for example

$$N = 8.$$

The FFT output contains the Fourier coefficients in the usual order

$$m = 0, 1, 2, 3, 4, -3, -2, -1.$$

Therefore, the corresponding physical angular frequencies are

$$\omega_m = 0, \Delta\omega, 2\Delta\omega, 3\Delta\omega, 4\Delta\omega, -3\Delta\omega, -2\Delta\omega, -\Delta\omega.$$

In Python or C, these values are stored in array positions

$$j = 0, 1, 2, 3, 4, 5, 6, 7.$$

Thus,

array position j	0	1	2	3	4	5	6	7
discrete frequency m	0	1	2	3	4	-3	-2	-1

The important point is that

$$j \neq m$$

in general.

For example, the value stored at array position

$$j = 7$$

corresponds to

$$m = -1,$$

and therefore to the physical frequency

$$\omega = -\Delta\omega.$$

It does **not** correspond to

$$m = 7.$$

For an even N , the mapping from a zero-based FFT array position j to the discrete frequency coordinate m can be written as

$$m = \begin{cases} j, & 0 \leq j \leq N/2, \\ j - N, & N/2 < j < N. \end{cases}$$

Thus, the FFT array position j describes where a coefficient is stored, while m describes the discrete frequency represented by that coefficient.

FFT shift

Sometimes the FFT output is rearranged using an operation such as `fftshift`. After shifting, the coefficients are usually displayed in increasing frequency order:

$$m = -\frac{N}{2}, \dots, -2, -1, 0, 1, 2, \dots, \frac{N}{2} - 1.$$

For $N = 8$, this becomes

$$m = -4, -3, -2, -1, 0, 1, 2, 3.$$

The physical frequency is still determined by

$$\omega_m = m\Delta\omega.$$

Only the **array ordering** has changed.

This illustrates why it is useful to keep the concepts separate:

$$\boxed{\text{physical coordinate} \longrightarrow \text{discrete coordinate} \longrightarrow \text{array storage}}$$

or, more specifically,

$$\boxed{t \longleftrightarrow n = \frac{t}{\Delta t} \longleftrightarrow k}$$

and

$$\boxed{\omega \longleftrightarrow m = \frac{\omega}{\Delta \omega} \longleftrightarrow j.}$$

The discrete coordinates n and m describe the mathematics. The array positions k and j describe how those values happen to be stored in MATLAB, Python, C, or another programming language.